CS 6515: Graduate Algorithms

Lecture 4: Tree Dynamic Programming, Knapsack, and an FPTAS

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW 1 due today.
2. HW 2 out in a couple of days (due on Thurs Sep 17).

2 Recap and plan

Last time we used dynamic programming for the largest contiguous sum. The central design question was:

What information must a subproblem remember so that it can be extended?

Today we will first use DP for longest common subsequence and then study use two different state spaces: subtrees of a rooted tree and prefixes paired with a target value. We will then preview approximation algorithms by turning the knapsack DP into an FPTAS.

3 Longest common subsequence

Input. Sequences

$$A = a_1 a_2 \cdots a_n, \qquad B = b_1 b_2 \cdots b_m.$$

A *subsequence* is obtained by deleting zero or more elements without changing the order of those retained. Unlike a substring, its elements need not occupy consecutive positions.

Goal. Find the maximum length of a sequence that is a subsequence of both A and B.

Example. If $A =$ `ABCDEFG` and $B =$ `BCDGKAB`, then `BCDG` is a common subsequence of length 4.

Definition (prefix subproblem). For $0 \le i \le n$ and $0 \le j \le m$, let $D[i, j]$ be the length of a longest common subsequence of $a_1 \cdots a_i$ and $b_1 \cdots b_j$.

The empty prefix gives the base cases

$$D[0, j] = D[i, 0] = 0.$$

For $i, j \ge 1$,

$$D[i, j] = \begin{cases} D[i-1, j-1] + 1, & a_i = b_j, \\ \max\{D[i-1, j], D[i, j-1]\}, & a_i \neq b_j. \end{cases}$$

Why the recurrence works. If $a_i = b_j$, some longest common subsequence can end with this common symbol; what precedes it is an LCS of the two shorter prefixes. If $a_i \neq b_j$, a common subsequence cannot use both final symbols, so at least one must be discarded.

The table has $(n+1)(m+1)$ entries, each filled in constant time. Thus the running time is $\mathcal{O}(nm)$ and the straightforward space usage is $\mathcal{O}(nm)$.

3.1 Recovering an actual LCS

Start at (n, m) and trace backward:

- if $a_i = b_j$, output that symbol and move to $(i-1, j-1)$;
- otherwise move to a neighboring state whose value equals $D[i, j]$.

The symbols are discovered in reverse order. The traceback takes $\mathcal{O}(n+m)$ time once the table is available.

Exercise. Draw the DP table for the two strings in the example and recover one LCS. Where can ties occur, and what do they mean?

DP takeaway. The hardest part is usually not filling the table; it is choosing a state that contains exactly the information needed by the recurrence. Next time we will practice this idea on trees and on the knapsack problem.

4 Maximum independent set in a tree

Let $T = (V, E)$ be an undirected tree. An *independent set* is a subset $S \subseteq V$ such that no edge has both endpoints in S.

Goal. Find an independent set of maximum cardinality.

Root the tree at an arbitrary vertex r. For a vertex v, let T_v denote the subtree rooted at v.

Definition (subproblem). Let $M[v]$ be the size of a largest independent set in T_v.

There are two possibilities at v:

- **include v:** then none of its children may be included, but each grandchild roots an independent remaining subproblem;
- **exclude v:** then each child roots an independent subproblem.

Thus

$$M[v] = \max \left\{ 1 + \sum_{u \in \text{grandchildren}(v)} M[u], \sum_{u \in \text{children}(v)} M[u] \right\}.$$

For a leaf v, this gives $M[v] = 1$.

Evaluation order. Run a postorder traversal, so every child and grandchild value is available before computing $M[v]$. Each vertex appears in the computation of only its parent and grandparent, so the total running time is $\mathcal{O}(|V|)$.

Exercise. Draw a rooted tree with at least eight vertices, compute every $M[v]$ bottom-up, and recover an optimal set. What decision must be recorded at each vertex?

4.1 An equivalent two-state formulation

Sometimes it is cleaner to explicitly record the decision at the root of each subtree. Define

$$I[v] = \text{best size in } T_v \text{ subject to including } v,$$
$$X[v] = \text{best size in } T_v \text{ subject to excluding } v.$$

Then

$$I[v] = 1 + \sum_{u \in \text{children}(v)} X[u],$$
$$X[v] = \sum_{u \in \text{children}(v)} \max\{I[u], X[u]\},$$

and the answer is $\max\{I[r], X[r]\}$.

Why this is useful. The state says exactly what constraint the parent needs to impose on a child. This pattern generalizes to many tree problems: create one state for each possible interaction across the edge to the parent.

Exercise. Prove that the one-state and two-state recurrences return the same optimum. Which formulation makes solution recovery easier?

5 The knapsack problem

There are n items. Item i has a positive integer size s_i and a positive integer value v_i. A knapsack has integer capacity B.

Goal. Choose $S \subseteq \{1, \ldots, n\}$ maximizing

$$\sum_{i \in S} v_i \qquad \text{subject to} \qquad \sum_{i \in S} s_i \leq B.$$

Example. Suppose the sizes are $(6, 3, 4, 2)$, the values are $(30, 14, 16, 9)$, and $B = 10$. Items 1 and 3 fit exactly and have total value 46.

Knapsack is NP-hard, so we do not expect an algorithm polynomial in the ordinary bit length of every input unless $\mathsf{P} = \mathsf{NP}$. We can, however, solve it by a dynamic program whose time is polynomial in the *numeric values* of the input.

5.1 A value-indexed dynamic program

Let

$$V_{\text{sum}} = \sum_{i=1}^{n} v_i.$$

Definition (subproblem). For $0 \le i \le n$ and $0 \le v \le V_{\text{sum}}$, let $D[i, v]$ be the minimum total size of a subset of the first i items having total value exactly v. If no such subset exists, set $D[i, v] = +\infty$.

The base cases are

$$D[0, 0] = 0, \qquad D[0, v] = +\infty \quad (v > 0).$$

For $i \ge 1$,

$$D[i, v] = \begin{cases} \min\{D[i-1, v],\ s_i + D[i-1, v - v_i]\}, & v \ge v_i, \\ D[i-1, v], & v < v_i. \end{cases}$$

The answer is the largest v for which $D[n, v] \le B$.

Correctness. Any subset of the first i items either excludes item i, giving the first candidate, or includes item i, leaving value $v - v_i$ to be realized by the first $i - 1$ items. The recurrence takes the smaller feasible size.

The table has $\mathcal{O}(nV_{\text{sum}})$ entries and constant work per entry. Since $V_{\text{sum}} \le n \max_i v_i$, the running time is

$$\mathcal{O}(nV_{\text{sum}}) = \mathcal{O}(n^2 \max_i v_i).$$

5.2 Why this is pseudo-polynomial

Writing an integer x in binary takes $\Theta(\log x)$ bits. The input length contains terms such as $\log v_i$ and $\log B$, but the DP time depends on the values themselves. For example, $v_i = 2^k$ needs only $k + 1$ bits to write, while an iteration for every value up to v_i takes exponential time in k.

An algorithm polynomial in the numeric magnitudes of integer inputs, but not necessarily in their encoding length, is called *pseudo-polynomial*.

Exercise. Fill the value-indexed table for the example and recover the chosen items. Then design the familiar capacity-indexed recurrence and compare its running time.

6 Preview: an FPTAS for knapsack

The pseudo-polynomial DP is slow only because its value axis can be enormous. The natural idea is to round values so that the table becomes small. Rounding must be coarse enough to improve the running time but fine enough to control the total loss.

Approximation language. For a maximization problem, a $(1-\varepsilon)$-approximation returns a feasible solution of value at least $(1-\varepsilon)\,\mathrm{OPT}$, where $0 < \varepsilon < 1$. A polynomial-time approximation scheme (PTAS) has polynomial running time for every fixed ε. A fully polynomial-time approximation scheme (FPTAS) has running time polynomial in both the input length and $1/\varepsilon$.

First discard items with $s_i > B$, since they can never be selected. If no item remains, return the empty set. Otherwise let

$$V = \max_i v_i.$$

Because an item attaining V is feasible by itself, $V \le \mathrm{OPT}$.

Choose the scaling unit

$$K = \frac{\varepsilon V}{n}$$

and replace each value by the nonnegative integer

$$\widehat{v}_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

Run the value-indexed knapsack DP using the scaled values $\widehat{v}_i$, and let $\widehat{S}$ be the feasible subset with maximum total scaled value.

Theorem (knapsack FPTAS). For every $0 < \varepsilon < 1$, the scaling algorithm returns a feasible set $\widehat{S}$ such that

$$v(\widehat{S}) \ge (1-\varepsilon)\,\mathrm{OPT}\,.$$

Its running time is $\mathcal{O}(n^3/\varepsilon)$ under the same unit-cost arithmetic convention as the pseudo-polynomial DP.

Running time. For every item,

$$\widehat{v}_i \le \frac{v_i}{K} \le \frac{V}{K} = \frac{n}{\varepsilon}.$$

Hence

$$\sum_{i=1}^{n} \widehat{v}_i \le \frac{n^2}{\varepsilon},$$

and the value-indexed DP takes

$$\mathcal{O}\left(n \sum_i \widehat{v}_i\right) = \mathcal{O}(n^3/\varepsilon)$$

time. This is polynomial in both the encoded input length and $1/\varepsilon$.

6.1 Approximation guarantee

Rounding down loses less than K on each selected item:

$$v_i - K < K\widehat{v}_i \leq v_i.$$

Let S^* be an optimal solution. Because $\widehat{S}$ maximizes scaled value among all feasible sets,

$$\begin{aligned} v(\widehat{S}) &\geq K \sum_{i \in \widehat{S}} \widehat{v}_i \\ &\geq K \sum_{i \in S^*} \widehat{v}_i \\ &\geq \sum_{i \in S^*} (v_i - K) \\ &\geq \mathrm{OPT} - nK \\ &= \mathrm{OPT} - \varepsilon V \\ &\geq (1 - \varepsilon)\,\mathrm{OPT}\,. \end{aligned}$$

Thus the total rounding loss is at most $nK = \varepsilon V$, and $V \leq \mathrm{OPT}$ converts this additive loss into the desired multiplicative guarantee.

What to notice. The approximation algorithm is still the same dynamic program. We changed the numbers used to index its states, trading a controlled amount of objective value for a polynomial number of states. Later, in the approximation-algorithms unit, we will develop this design pattern and its analysis more systematically.

Exercise. For the running example, take $\varepsilon = 1/2$. Compute K and every scaled value $\widehat{v}_i$, run the scaled DP, and compare its returned value with $(1 - \varepsilon)\,\mathrm{OPT}$. Is the proven bound tight on this input?

Takeaway. DP can make an exponential search manageable by merging partial solutions that have the same relevant state. When the exact state space is too large, rounding can compress it while preserving a provable approximation. Next time we continue DP with uncertainty and finite-horizon decision making.